Control Tower Workflow Guide

Part 2 of 7: n8n basics, then every node in CT-02 (State & Audit Writer) and CT-03 (Dispatch Action)

Source: your uploaded exports **CT-02 Sub - State & Audit Writer** (12 nodes) and **CT-03 Sub - Dispatch Action** (7 nodes). Everything here comes from the JSON. Sample data is marked **illustrative**. Where something depends on a live run or an outside service, it says so.

In this part	Pages (approx.)
1. Six n8n ideas you need first (items, JSON, expressions, sub-workflows)	2-4
2. CT-02 State & Audit Writer: big picture, the 9 working nodes, a worked example	4-10
3. CT-03 Dispatch Action: big picture, the 5 working nodes, retry arithmetic	10-14
4. Self-check questions	14

Plain English
CT-02 and CT-03 are the two *helper* workflows the others lean on. Learn them first and the big workflow (CT-01, Parts 3-4) becomes much easier, because CT-01 mostly consists of 'prepare a message, hand it to CT-02' repeated again and again.

1. Six n8n ideas you need first

1.1 An item: one envelope of information

In n8n, information travels between nodes as **items**. Think of an item as one envelope, or one row in a spreadsheet. A node receives one or more envelopes, does its job, and hands out new envelopes. If a database search finds 50 incidents, the next node receives 50 items.

1.2 JSON: labelled boxes inside the envelope

JSON is just a way of writing information as **label: value** pairs. The label is the **field name**. Field names must match exactly: `incident_id` and `incidentId` are different labels, and a node looking for the wrong one finds nothing.

```
{
  "incident_id": "sample-id-123",     <- text value
  "retry_count": 1,                   <- number
  "ai_failed": false,                 <- true/false
  "metadata": { "note": "hello" },    <- a box inside a box
  "tags": ["a", "b"]                  <- a list
}
```

1.3 Passing data, and why it changes shape

The output of a node becomes the input of the next node along the connecting line. But nodes do not all behave the same way: some **add** fields (a database read adds the row), some **replace** the whole envelope (a Set node in 'raw' mode, used all over your system, throws away what came in and outputs only what you wrote), and some **filter** (a Code node that returns only some items).

1.4 Expressions: reading a box

Anything written inside `{{ ... }}` is an **expression**: a tiny instruction evaluated while the workflow runs.

Expression	Plain English	Where you will see it
`{{ $json.incident_id }}`	Open the envelope that just arrived and read the box called incident_id.	CT-02 'Database - Update Incident'
`$('Code - Validate Transition').item.json.audit`	Walk back to the desk called 'Code - Validate Transition', find the envelope that matches this one, and read its 'audit' box.	CT-02 'Set - Row: Audit Log'
`$('Loop - Each Due Retry').first().json`	Go back to the loop node and read its first output envelope.	CT-05
`$execution.id`, `$workflow.name`	Built-in facts: the number of this run, and the name of this workflow.	Every 'Prepare ... Update' node

1.5 Many items, no items, missing fields

- **Many items:** most nodes process each envelope in turn. CT-02's code loops over all incoming items; CT-05 uses a Loop node to take one incident at a time.
- **No items:** if a node outputs nothing, the nodes after it simply do not run. That is normal for 'search found nothing'. Some nodes in your system switch on *Always Output Data* (for example CT-01's duplicate check and CT-03's config lookup) so that one **empty** envelope still moves on and the workflow can make a decision.
- **Missing field:** reading a box that is not there gives an empty value rather than a crash, *unless* code then relies on it. For example CT-03 deliberately throws an error if the channel setting is missing.

1.6 Sub-workflows: phoning another workflow

An **Execute Workflow** node is a phone call to another workflow. The other workflow starts with an **Execute Workflow Trigger**, the phone that rings. By default the caller **waits**, and what the called workflow's *last node* outputs comes back as the answer. If the called workflow fails, the call fails, unless the caller has an **error output** switched on (CT-01 and CT-05 do this on their CT-03 call, so a failed dispatch can be retried instead of crashing). A call can also be made **fire-and-forget** (not waiting): that is how CT-02 would call CT-07.

1.7 Passed versus saved

Items exist only while a run is happening. Data is **saved** only if a node writes it somewhere permanent, such as a Supabase table. In your system the permanent memory is the database: the `incidents` table is the current status of each case, and `audit_logs` is the history book.

2. CT-02 Sub - State & Audit Writer

Workflow id 7krMfRZepEVi3L9V. Active. No error workflow attached. 12 nodes: 9 working nodes plus 3 sticky notes (Overview, Allowed state changes, HubSpot hook).

What it is for

CT-02 is the **file clerk**. Its own note says it is the ONLY place that changes an incident's status. Whoever wants a status change (CT-01, CT-04, CT-05, CT-06) hands CT-02 a request and CT-02 does three things: **check** the move is legal, **update** the incident, **log** it. Then it replies with a short receipt.

Who starts it and what they must send

The trigger is set to **passthrough** (accept all data), so the callers' envelopes arrive exactly as sent. Every caller in your system builds the same shape:

Field the caller sends	Meaning	Required by the code?
incident_id	Database id of the incident (the 'id' column)	Yes - error if missing
previous_state / new_state	Where the incident is now, and where it should go	Yes - error if either missing
event	Short label for the history book, e.g. DISPATCH_FAILED. If absent, CT-02 invents one like PROCESSING_TO_COMPLETED	No
status	SUCCESS, WARNING, FAILURE or INFO. Defaults to SUCCESS	No
event_id, execution_id, workflow_name, actor, payload_hash, error_message	Extra context for the history row. actor defaults to 'system'	No
patch	Extra columns to change on the incident (e.g. retry_count, next_retry_at, last_error)	No
metadata	Free notes stored on the history row	No
context	Anything the caller wants echoed back in the receipt	No

The allowed state changes, as coded

From	May move to
RECEIVED	VALIDATING, ESCALATED, FAILED
VALIDATING	AI_PROCESSING, REJECTED, ESCALATED, FAILED
AI_PROCESSING	CLASSIFIED, ESCALATED, FAILED
CLASSIFIED	PROCESSING, AWAITING_APPROVAL, ESCALATED, FAILED
AWAITING_APPROVAL	APPROVED, REJECTED, ESCALATED, FAILED
APPROVED	PROCESSING, ESCALATED, FAILED
PROCESSING	COMPLETED, RETRYING, ESCALATED, FAILED
RETRYING	PROCESSING, ESCALATED, FAILED

From	May move to
ESCALATED	RETRYING, FAILED
FAILED	RETRYING
COMPLETED, REJECTED	nothing (final)

Staying in the same state (previous = new) is always allowed. That is how CT-02 records 'audit-only' events such as EVENT_RECEIVED or AI_FALLBACK without moving the incident.

Path through CT-02

```
Trigger -> Code: Validate Transition -> Set: Row Incident Update -> DB: Update Incident
        -> Set: Row Audit Log -> DB: Write Audit Log -> IF: Final State Reached?
                                                          |-- true  -> (CT-07 call, DISABLED) -> Set: Return Result
                                                          |-- false -> Set: Return Result
```

The nodes, one by one

Node 1: Trigger - Receive State Change

Actual node type: n8n-nodes-base.executeWorkflowTrigger (v1.1), inputSource = passthrough

A. What is this node?	The 'phone that rings'. It starts CT-02 whenever another workflow calls it.
B. Why is it needed?	Without a trigger, a workflow cannot be called by another one. 'Passthrough' means it accepts whatever data the caller sends, with no fixed form to fill in.
C. Imagine it as...	A reception desk that accepts any envelope and passes it on unopened.
D. What enters	One or more items from the caller, e.g. from CT-01: incident_id, previous_state, new_state, event, status, patch, and so on (table above).
E. What happens inside	Nothing is changed. It only hands the envelope(s) on.
F. What leaves	The same items, unchanged.
G. Where it goes next	**Code - Validate Transition**.
H. If something goes wrong	No error handling of its own. If the caller sends nothing useful, the failure appears in the next node.
I. Why it matters	It is the single entrance to the system's only status-changing machinery.

Node 2: Code - Validate Transition

Actual node type: n8n-nodes-base.code (v2), JavaScript

A. What is this node?	A small program that checks each request and prepares three ready-made packages: an **update** for the incident, an **audit** row for the history book, and an **echo** (the receipt).
B. Why is it needed?	This is the rule book. It stops impossible moves (for example COMPLETED back to PROCESSING) before anything reaches the database.
C. Imagine it as...	A strict clerk with a rule sheet who stamps 'approved' or 'refused' and pre-fills three forms.
D. What enters	Every incoming item. Important fields: incident_id, previous_state, new_state, plus the optional ones listed earlier.
E. What happens inside	For each item: (1) throw an error if incident_id, previous_state or new_state is missing; (2) throw an error if new_state is not a known state; (3) throw an error if previous_state is different from new_state and new_state is not in the allowed list for previous_state; (4) build **update** = the patch fields, then id, state and updated_at (listed last so a patch can never overwrite them), and add resolved_at when the new state is COMPLETED or REJECTED; (5) build **audit** with 12 fields (defaults: status SUCCESS, actor 'system', metadata {}); (6) build **echo** = {ok: true, incident_id, new_state, context}.
F. What leaves	One item per input, shaped {update, audit, echo}. See the example below.
G. Where it goes next	**Set - Row: Incident Update**.
H. If something goes wrong	Any thrown error stops CT-02 with that message. The caller's call fails; whether anything catches it depends on the caller (CT-01's and CT-05's CT-02 calls have no error output). The database is not touched because the error happens first.

I. Why it matters	Protects data quality. Every incident history is guaranteed to follow legal steps, so reports and the watchdog can trust the states.

Example of what goes in and what comes out

Illustrative example (not a real run)
A dispatch to Slack failed in CT-01, so CT-01 asks CT-02 to move incident 'sample-id-123' from PROCESSING to RETRYING. Values are made up; the field names are the real ones.

Input (what CT-01's 'Code - Build Retry Update' sends):


```
{ "incident_id": "sample-id-123", "event_id": "EVT-1001", "execution_id": "5120",
  "workflow_name": "CT-01 Main - Event Intake & Orchestration",
  "event": "DISPATCH_FAILED", "previous_state": "PROCESSING", "new_state": "RETRYING",
  "status": "FAILURE", "error_message": "channel_not_found", "actor": "system",
  "payload_hash": "ab12...", "metadata": {},
  "patch": { "retry_count": 1, "next_retry_at": "2026-10-10T09:01:00.000Z",
             "last_error": "channel_not_found" },
  "context": null }
```


Output (one item with three packages):


```
{ "update": { "retry_count": 1, "next_retry_at": "2026-10-10T09:01:00.000Z",
              "last_error": "channel_not_found", "id": "sample-id-123",
              "state": "RETRYING", "updated_at": "2026-10-10T09:00:03.411Z" },
  "audit":  { "incident_id": "sample-id-123", "event_id": "EVT-1001", "execution_id": "5120",
              "workflow_name": "CT-01 Main - Event Intake & Orchestration",
              "event": "DISPATCH_FAILED", "previous_state": "PROCESSING", "new_state": "RETRYING",
              "status": "FAILURE", "error_message": "channel_not_found", "actor": "system",
              "payload_hash": "ab12...", "metadata": {} },
  "echo":   { "ok": true, "incident_id": "sample-id-123", "new_state": "RETRYING", "context": null } }
```


The patch fields were moved into **update**, then id, state and a fresh updated_at were added. The same facts were also copied into **audit**, and **echo** is the small receipt that will go back to the caller.

Audit flag
Possible weakness (design, not a crash): the legal-move check compares against the *previous_state the caller claims*, not against the state actually stored in the database. CT-02 never reads the incident's current state, and the database update (Node 4) matches on id alone. If a caller is out of date or wrong, an illegal move could still be written. Whether this can happen in practice cannot be confirmed without live testing. A fix is suggested in Part 7.

Node 3: Set - Row: Incident Update

Actual node type: n8n-nodes-base.set (v3.4), mode = raw JSON, jsonOutput = {{ $json.update }}

A. What is this node?	An 'Edit Fields' node used in raw mode: it writes out exactly the JSON you give it.
B. Why is it needed?	The next node saves whatever it receives as columns. This node makes sure it receives **only** the update package, not audit or echo.
C. Imagine it as...	A clerk who pulls one form out of a folder and puts only that form on the desk.
D. What enters	The {update, audit, echo} item from Node 2.
E. What happens inside	Reads the 'update' box and outputs its contents as the whole item.
F. What leaves	{ retry_count: 1, next_retry_at: ..., last_error: ..., id: 'sample-id-123', state: 'RETRYING', updated_at: ... } (illustrative). The audit and echo packages are **gone from this item**; later nodes fetch them again from Node 2 by name.

G. Where it goes next	**Database - Update Incident**.
H. If something goes wrong	No error handling configured. If 'update' were missing the item would be empty, but Node 2 always creates it.
I. Why it matters	Keeps the database write clean and predictable.

Node 4: Database - Update Incident

Actual node type: n8n-nodes-base.supabase (v1), operation = update, table = incidents

A. What is this node?	A database action: change an existing row in the Supabase table called **incidents** (the current status board).
B. Why is it needed?	This is the moment the incident's state actually changes in permanent storage. Before this, everything was only a plan.
C. Imagine it as...	Erasing and rewriting the status on a case file in the filing cabinet.
D. What enters	The update package from Node 3. The key field is **id**, used to find the row.
E. What happens inside	Finds the single row where **id** equals the incoming id (filter: id eq {{ $json.id }}). Because data mode is 'autoMapInputData', **every field in the item becomes a column to write**: state, updated_at, resolved_at (if present) and any patch fields (retry_count, next_retry_at, last_error, rejection_reason, ai_*, etc.). The id field is used for the match as well.
F. What leaves	Supabase normally returns the updated row. The exact output cannot be confirmed without running the workflow. The next node does not use it.
G. Where it goes next	**Set - Row: Audit Log**.
H. If something goes wrong	No retry and no error output on this node. If the database is down, or a patch contains a column name that does not exist in the table, the node fails and so does CT-02, and so does the caller's call. Because the history row (Node 6) comes *after* this node, a failure here means nothing is logged by CT-02 for that change.
I. Why it matters	This is the system's single source of truth. Dashboards, the watchdog and the recovery endpoint all read this table.

Audit flag
Possible weakness: the update (Node 4) and the audit log (Node 6) are two separate database writes with no transaction around them. If the second fails after the first succeeds, the incident has moved but the history is missing a row. Whether the audit_logs insert can fail depends on table rules not visible here.

Node 5: Set - Row: Audit Log

Actual node type: n8n-nodes-base.set (v3.4), mode = raw, jsonOutput = {{ $('Code - Validate Transition').item.json.audit }}

A. What is this node?	Another raw Set node. This one goes **back to Node 2** and fetches the 'audit' package.
B. Why is it needed?	Node 4's output replaced the envelope, so the audit data must be fetched by name from an earlier node. '.item' means 'the matching envelope for this one', so with many items each update pairs with its own audit row.
C. Imagine it as...	Walking back to an earlier desk to pick up a form you set aside.
D. What enters	The result of Node 4 (not actually used).
E. What happens inside	Looks up the Node 2 item that this item descended from (n8n tracks this with 'pairedItem', which Node 2 set deliberately) and outputs its audit object.
F. What leaves	The audit object only, e.g. {incident_id, event_id, execution_id, workflow_name, event: 'DISPATCH_FAILED', previous_state: 'PROCESSING', new_state: 'RETRYING', status: 'FAILURE', ...} (illustrative).
G. Where it goes next	**Database - Write Audit Log**.
H. If something goes wrong	No handling configured. If the pairing link were lost the expression would fail; Node 2 sets it explicitly to prevent that.
I. Why it matters	Guarantees each history row matches its own update when several incidents are processed at once.

Node 6: Database - Write Audit Log

Actual node type: n8n-nodes-base.supabase (v1), operation = create (default), table = audit_logs

A. What is this node?	A database action that **adds a new row** to **audit_logs**, the history book.
B. Why is it needed?	An audit log answers 'what happened, when, by whom, and why' for every change. Rows are only ever added here, not edited.
C. Imagine it as...	The logbook kept at a security desk: one line per event, never erased.
D. What enters	The audit object (incident_id, event_id, execution_id, workflow_name, event, previous_state, new_state, status, error_message, actor, payload_hash, metadata).
E. What happens inside	Inserts one row; each field becomes a column (autoMapInputData). The column names must exist in audit_logs.
F. What leaves	Supabase normally returns the inserted row, including new_state (the next node relies on this). Cannot be confirmed without running.
G. Where it goes next	**IF - Final State Reached?**.
H. If something goes wrong	No retry, no error output. A failure stops CT-02 and the caller's call fails.
I. Why it matters	This is the evidence trail for every state change: the basis for investigations and for proving what the automation did.

Node 7: IF - Final State Reached?

Actual node type: n8n-nodes-base.if (v2.2), loose type checking

A. What is this node?	A yes/no question node with two exits: **true** (top) and **false** (bottom).
B. Why is it needed?	Decides whether this change is an 'outcome' worth reporting to the CRM.
C. Imagine it as...	A sorter at the end of the belt: 'finished or not?'
D. What enters	The row just written to audit_logs. It looks at the **new_state** field.
E. What happens inside	Evaluates {{ ['COMPLETED','REJECTED','ESCALATED','FAILED'].includes($json.new_state) }}: is the new state one of those four?
F. What leaves	The same item, sent out of the true or false exit. Nothing is changed.
G. Where it goes next	**True:** Sub-workflow - Sync Outcome to HubSpot (CT-07). **False:** Set - Return Result. (After the CT-07 node, the path also continues to Set - Return Result, so both branches meet again.)
H. If something goes wrong	If new_state were missing (for example if the database returned an unexpected shape) the answer would be false and the HubSpot step would be skipped silently.
I. Why it matters	Means HubSpot only hears about finished or abandoned incidents, not every intermediate step.

Node 8: Sub-workflow - Sync Outcome to HubSpot (CT-07)

Actual node type: n8n-nodes-base.executeWorkflow (v1.2), mode = each, waitForSubWorkflow = false, **disabled = true**, onError = continueRegularOutput

A. What is this node?	A phone call to workflow **CT-07 Sub - HubSpot Outcome Sync** (id Vv0K3JHniLnTpKqV).
B. Why is it needed?	Designed to start the HubSpot note-taking in the background without making CT-02 wait.
C. Imagine it as...	Dropping a note in someone else's in-tray and walking away without waiting for a reply.
D. What enters	The item from the IF node (the audit row, which contains incident_id and new_state). CT-07 only reads **incident_id**.
E. What happens inside	**As exported this node is switched off.** In n8n a disabled node does nothing but pass its input straight through. If switched on, it would start CT-07 once per item ('each') and **not wait** for the answer.
F. What leaves	While disabled: the input unchanged. When enabled and not waiting: n8n continues immediately, so the HubSpot work cannot slow CT-02. What CT-07 returns would not be used.
G. Where it goes next	**Set - Return Result**.
H. If something goes wrong	onError = continueRegularOutput, so even a failure launching CT-07 would not stop CT-02. Since it does not wait, a failure *inside* CT-07 will not appear in CT-02 either.
I. Why it matters	Keeps the CRM informed without risking the core system. But see the flag.

Audit flag
Confirmed configuration finding: this node has `"disabled": true`, so CT-07 is **never started**. The CT-02 sticky note and CT-07's own note both describe it as live, so this is probably leftover from testing. Also, the call's input mapping is 'define below' with no fields while CT-07's trigger is 'passthrough'. Whether CT-07 receives incident_id correctly cannot be confirmed without a live test after enabling.

Node 9: Set - Return Result

Actual node type: n8n-nodes-base.set (v3.4), mode = raw, jsonOutput = {{ $('Code - Validate Transition').item.json.echo }}

A. What is this node?	The last node. Because it is the last node, **what it outputs is what CT-02 sends back to whoever called it**.
B. Why is it needed?	Gives the caller a short, predictable receipt, regardless of what the database nodes returned.
C. Imagine it as...	Handing back a stamped receipt: 'done, new state is X'.
D. What enters	Arrives from either exit of the IF node (via CT-07 node on the true side).
E. What happens inside	Fetches the 'echo' package prepared in Node 2.
F. What leaves	{ ok: true, incident_id: 'sample-id-123', new_state: 'RETRYING', context: null } (illustrative). **context** is whatever the caller sent as context. CT-05 uses this: it sends the incident row as context when it sets PROCESSING, and uses it again when sending a Slack stale alert.
G. Where it goes next	Nothing: CT-02 ends here and the receipt returns to the calling workflow.
H. If something goes wrong	If an earlier node failed, this node never runs and no receipt is returned; the caller's call fails instead.
I. Why it matters	Gives callers confidence ('ok: true') that the state change and the log both succeeded.

Worked example: one RETRYING request, start to finish (simulation)

Step	What happens	Data in the envelope
1	CT-01 calls CT-02 after Slack failed	The request with new_state RETRYING and the patch
2	Node 2 checks PROCESSING -> RETRYING	Legal (it is in PROCESSING's list), so it builds update, audit, echo
3	Node 3 keeps only the update package	id, state, retry_count, next_retry_at, last_error, updated_at
4	Node 4 changes the incident row	The incidents row now shows RETRYING with a retry time one minute ahead
5	Nodes 5-6 write the history row	audit_logs gains an event named DISPATCH_FAILED with status FAILURE
6	Node 7 asks 'final?' New state RETRYING is not on the list	False exit: no HubSpot call
7	Node 9 returns the receipt	{ ok: true, new_state: 'RETRYING' }

3. CT-03 Sub - Dispatch Action

Workflow id JfhHBm4rMmgkcByL. Active. No error workflow attached. 7 nodes: 5 working nodes plus 2 sticky notes (Overview, Failure Demo).

What it is for

CT-03 is the **messenger**. 'Dispatching the action' in this system means one thing: **posting a formatted alert to a Slack channel**. The 'Also notify' line in that message is plain text; the JSON shows no node that contacts those other departments separately.

Who calls it

- **CT-01**, node 'Sub-workflow - Dispatch Action (CT-03)', for the first attempt (attempt = 1).
- **CT-05**, node 'Sub-workflow - Re-Dispatch Action (CT-03)', for each retry (attempt = retry_count + 1).

Both send `{ incident, attempt }`. Both wait for the reply and both have **error output** switched on, so a CT-03 failure is routed to their retry logic rather than crashing them.

Path through CT-03

```
Trigger -> Database: Get Dispatch Config -> Code: Build Dispatch Message
        -> Slack: Post Incident Alert -> Set: Return Result
```

Node 10: Trigger - Receive Dispatch Request

Actual node type: n8n-nodes-base.executeWorkflowTrigger (v1.1), inputSource = passthrough

A. What is this node?	The phone that rings for CT-03.
B. Why is it needed?	Lets CT-01 and CT-05 call CT-03.
C. Imagine it as...	A reception desk that accepts any envelope.
D. What enters	{ incident: {...}, attempt: n }. The incident object carries priority, incident_no, department, event_type, entity_id, severity, risk_score, ai_summary, ai_recommended_action, notify_departments, sla_due.
E. What happens inside	Passes it on unchanged.
F. What leaves	The same item.
G. Where it goes next	**Database - Get Dispatch Config**.
H. If something goes wrong	None of its own.
I. Why it matters	Entry point for the only external 'action' the system performs.

Node 11: Database - Get Dispatch Config

Actual node type: n8n-nodes-base.supabase (v1), operation = getAll, table = system_config, returnAll = true, alwaysOutputData = true

A. What is this node?	Reads settings from a table called **system_config** (a settings notebook).
B. Why is it needed?	Keeps the Slack channel and the failure-demo switch in the database so they can be changed without editing the workflow.
C. Imagine it as...	Looking up two entries in a settings notebook.
D. What enters	The caller's item (it is not used for the lookup).
E. What happens inside	Looks for rows where the **key** column is 'dispatch_channel_id' or 'simulate_slack_failure'. The JSON gives two conditions and no 'match type', so the lookup only works if the node's default means 'match ANY condition'. If it meant 'match ALL', nothing could ever match both. n8n's default for this node is any-condition, but confirm with a live test.
F. What leaves	Up to two items, each like { key: 'dispatch_channel_id', value: 'C0...' } (illustrative). Because 'Always Output Data' is on, if nothing is found you still get one empty item.
G. Where it goes next	**Code - Build Dispatch Message**.
H. If something goes wrong	No retry. If Supabase is unreachable the node fails and so does CT-03. If rows are missing, the next node throws a clear error.
I. Why it matters	Allows operations staff to switch the alert channel, or switch on the demo failure, with a database edit.

Node 12: Code - Build Dispatch Message

Actual node type: n8n-nodes-base.code (v2), JavaScript

A. What is this node?	A small program that assembles the Slack message and picks the channel.
B. Why is it needed?	Separates 'what to say and where' from 'sending it'.
C. Imagine it as...	A secretary typing the message before the messenger leaves.
D. What enters	The config rows from Node 11, plus the original request, read by name from 'Trigger - Receive Dispatch Request'.
E. What happens inside	(1) Turns the rows into a lookup: key to value. (2) Reads the request; throws 'CT-03 input must contain { incident }' if there is no incident. (3) If simulate_slack_failure is the text 'true', replaces the channel with `C0000SIMFAIL`, an invalid id, so that Slack will refuse it (the live failure demo). (4) Throws an error if there is still no channel id. (5) Builds a multi-line message: priority, INC number, department, event type; entity, severity, risk score out of 100; the AI impact summary; the recommended action; departments to notify; SLA due time; and a footer with the attempt number and this execution's id. Missing AI text shows as 'n/a'.
F. What leaves	{ channel_id: 'C0...', text: ':rotating_light: *[P2] INC-42 | Logistics | SHIPMENT_DELAY* ...' } (illustrative). Only these two fields continue.
G. Where it goes next	**Slack - Post Incident Alert**.

H. If something goes wrong	Throws errors for a missing incident or a missing channel id. Either makes CT-03 fail, and the caller treats it exactly like a Slack failure and schedules a retry.
I. Why it matters	Delivers the information a responder needs, in one readable alert.

Illustrative example (not a real run)
Message text for a sample P2 Logistics incident (made-up values):

```
:rotating_light: *[P2] INC-42 | Logistics | SHIPMENT_DELAY*
Entity: `ORD-55412` | Severity: *HIGH* | Risk: 72/100
*AI impact:* Five-day delay puts a customer commitment at risk.
*Recommended action:* Contact carrier and update the customer.
*Also notify:* Customer Service, Sales, Finance
*SLA due:* 2026-10-10T11:00:00.000Z
_Dispatch attempt 1 | execution 5188_
```

Audit flag
Possible weakness: a config problem (missing channel row) is reported to the caller as an ordinary failure, so the incident goes into the retry cycle and is eventually ESCALATED, even though retrying cannot fix a missing setting. The error text is saved to last_error, so it can still be diagnosed.

Node 13: Slack - Post Incident Alert

Actual node type: n8n-nodes-base.slack (v2.3), post message to channel by id, retryOnFail = true, maxTries = 3, waitBetweenTries = 2000 ms

A. What is this node?	Posts a message to a Slack channel using the credential called 'Slack account 2'.
B. Why is it needed?	This is the actual dispatch: the moment humans are notified.
C. Imagine it as...	The messenger who walks into the room and announces the alert.
D. What enters	channel_id and text from Node 12. The channel is given by **id** (not name).
E. What happens inside	Asks Slack to post the text. If Slack answers with an error (such as channel_not_found), n8n **waits 2 seconds and tries again, up to 3 tries in total**.
F. What leaves	On success, Slack's reply describing the posted message (exact shape cannot be confirmed without running). It is not used afterwards.
G. Where it goes next	**Set - Return Result** (on success only).
H. If something goes wrong	No error output is configured on this node, so after the 3rd failure the node fails and **CT-03 itself fails**. That failure is what the caller catches through its error output and turns into RETRYING (CT-01) or another attempt (CT-05). Note CT-03 would also fail on a problem that retries cannot cure, such as an invalid channel.
I. Why it matters	Delivers the alert, and its deliberate failure is the trigger for the whole self-healing mechanism.

Node 14: Set - Return Result

Actual node type: n8n-nodes-base.set (v3.4), mode = raw, jsonOutput = { ok: true, incident: (the incident from the trigger) }

A. What is this node?	The last node, so its output is CT-03's reply to the caller.
B. Why is it needed?	Tells the caller 'it worked' and returns the incident it was given.
C. Imagine it as...	A 'delivered' receipt.
D. What enters	Slack's reply (ignored).
E. What happens inside	Reads the incident back from the trigger node and wraps it with ok: true.
F. What leaves	{ ok: true, incident: {...} } (illustrative).
G. Where it goes next	Nothing: returns to CT-01 (then 'Set - Prepare COMPLETED Update') or CT-05 (then 'Set - Prepare Self-Healed Update').
H. If something goes wrong	Only reached on success. A failure anywhere earlier prevents this and returns an error instead.
I. Why it matters	The caller only has to ask one question: did I get a normal reply or an error?

Retry arithmetic: how many times can Slack actually be tried?

The numbers below come from the code in CT-01 (Code - Build Retry Update), CT-05 (Code - Build Failure Update) and this workflow's Slack node.

Attempt	Who runs it	If it fails
1	CT-01 calls CT-03 (up to 3 Slack tries inside)	CT-01 sets RETRYING, retry_count = 1, next_retry_at = 1 minute ahead ('demo value; use 5 in production')
2	CT-05 retry: attempt = retry_count + 1 = 2	retry_count becomes 2; below the limit of 3, so RETRYING again, wait = 2 x 1 minute
3	CT-05 retry: attempt = 3	retry_count would be 3, which reaches max_retries (the row's value, or 3 if none), so the incident is ESCALATED with Gmail and Slack alerts

Good to know
So the system makes **3 dispatch attempts** in total, each with up to 3 Slack tries. That is up to 9 Slack calls before a human is contacted. The limit is 3 only if the incident row has no max_retries value of its own.

Good to know
Demo switch: setting `simulate_slack_failure = true` in system_config swaps the channel for an invalid id. Setting it back to false before the next CT-05 pass lets that retry succeed ('self-healing'). Leave it on and the three attempts run out and the incident escalates.

4. Self-check

Try to answer these without looking back. Answers are in the text above.

- Why does CT-02 use a Set node to fetch 'audit' from Node 2 instead of getting it from the database node before it?
- A caller asks CT-02 to move an incident from COMPLETED to PROCESSING. Which node refuses, and what happens to the incident row?
- Why does a disabled node not stop CT-02 from returning a receipt?
- In CT-03, what are the two different ways the workflow can fail, and what does the caller do in both cases?
- An incident is escalated after its third dispatch attempt. How many Slack calls could have been made?

Next: Part 3 covers CT-01, the intake and AI stage: webhook, validation, fingerprint, duplicate check, incident creation, HubSpot lookup and the AI agent.